Planning and reflection

Architecture 3: University teaching deck

How does an agent decide what to do next, and how does it learn from what it has already done?

Task decomposition, Chain-of-Thought, least-to-most, plan-and-solve, ReAct, FireAct, ETO, Self-Refine, Reflexion, TTRL, and R-Zero.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Planning

Explain why an agent needs a plan. Describe task decomposition and compare Chain-of-Thought, self-consistency, Tree of Thoughts, least-to-most prompting, and plan-and-solve prompting.

Action sequencing

Explain how ReAct interleaves thought, action, and observation. Compare ReAct by prompting, by supervised fine-tuning (FireAct), by reinforcement learning (ETO), and natively.

Reflection and self-improvement

Describe Self-Refine and Reflexion. Explain how Test-Time Reinforcement Learning (TTRL) and R-Zero let a model improve itself with little or no labeled data.

Source: Ch. 6, PDF pp. 297–346

Planning is the final core component

Last piece of a single agent

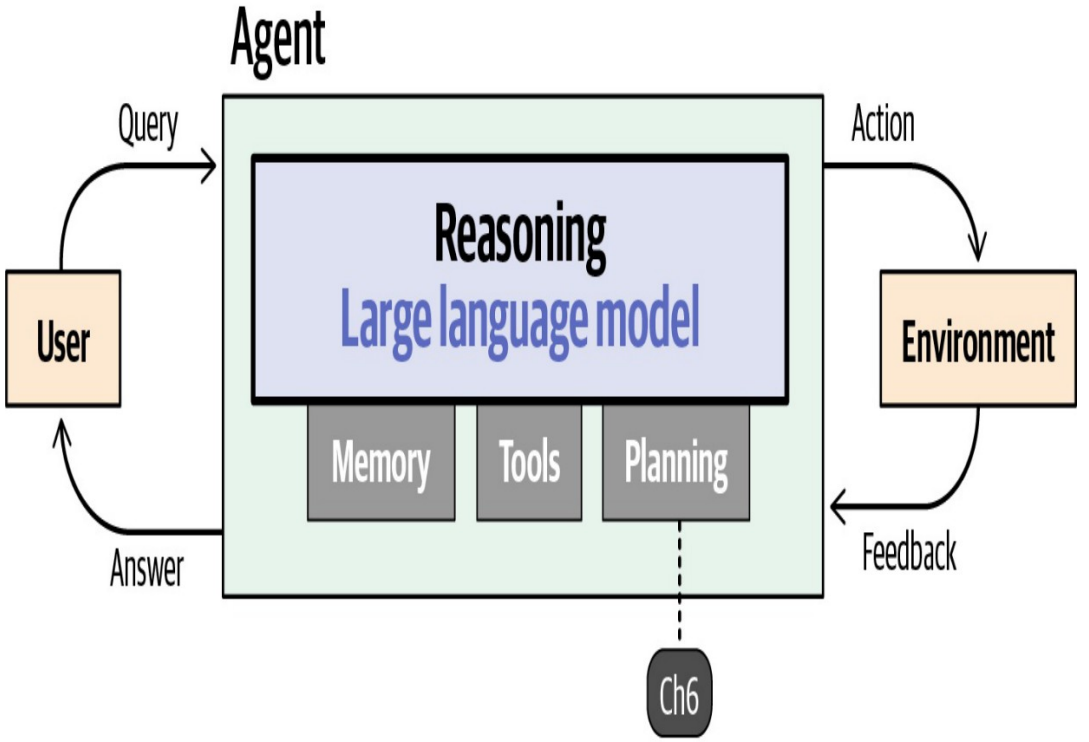
The agent so far has a reasoning LLM, tools, and memory. Planning and reflection complete it, as the highlighted module in the figure shows.

Reasoning enables planning

Chapter 3's reasoning LLMs can think through a chain of thought. That makes it possible to plan actions and to reflect on them.

A feedback loop

Planning and reflection go hand in hand. A feedback loop prevents the system from ending up in a local minimum.



Grootendorst & Alammar (2026). Fig. 6-1, PDF p. 298

Why agents need plans and reflection

A multi-step task

Adding a feature involves six typical steps: clarify requirements, analyze the codebase, design, implement, test, and update documentation.

Track the state

To execute them, the agent must first be aware of the steps and track its current state. Without a plan, how would it know what to do next?

Steps repeat

Implementing and testing may be repeated whenever a bug appears. Iterative loops are a fundamental part of what makes an agent.

Malleable plans

Initial plans stay open to change when the plan or its execution has a problem. Reflection on what was done and what could improve makes the agent autonomous.

Task decomposition splits a query into subtasks

First step in planning

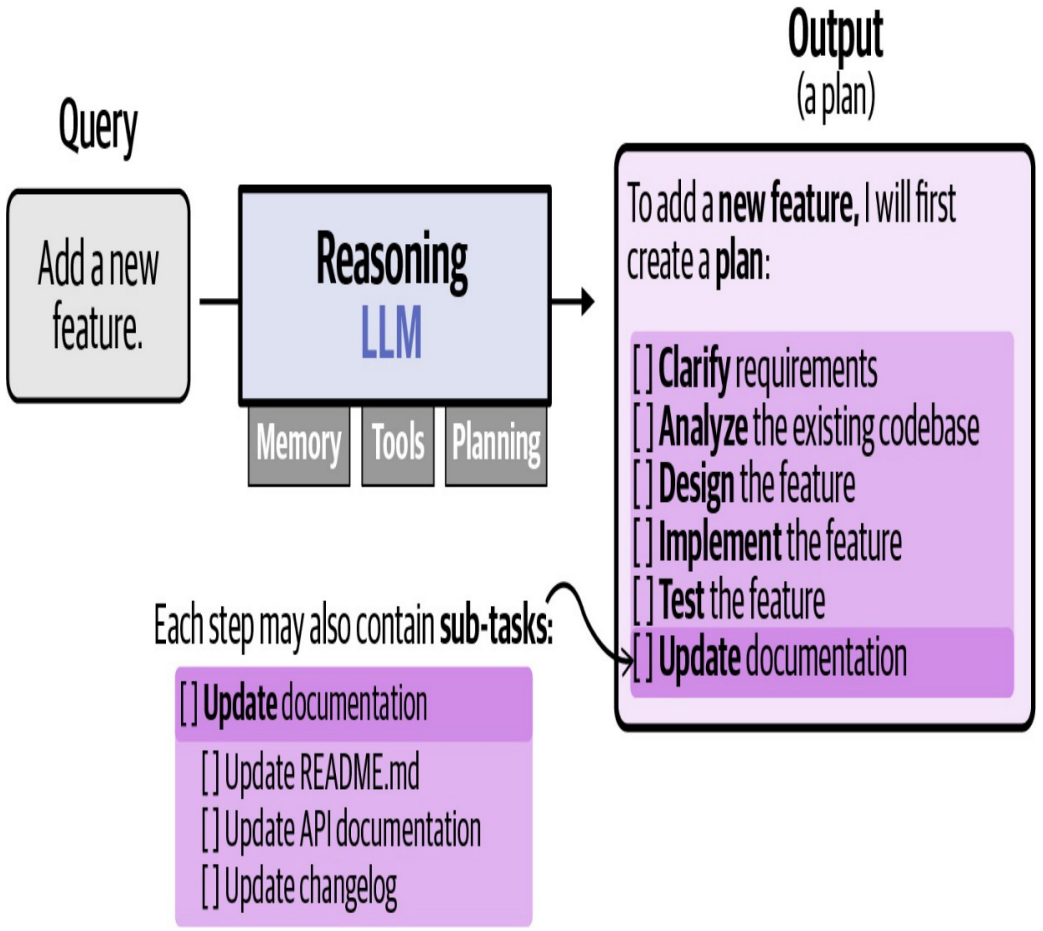
The first step is to decompose the initial query into subtasks. Asked to create a feature, the agent splits the request into smaller tasks.

Simplifies hard tasks

Task decomposition lets the agent simplify complicated tasks. The reasoning LLM produces a structured plan, as the figure shows.

Tasks within tasks

Each task may also have a set of its own subtasks to complete, so a plan can be a small tree rather than a flat list.



Grootendorst & Alammar (2026). Fig. 6-2, PDF p. 299

Plans and tool use work together

Plan first

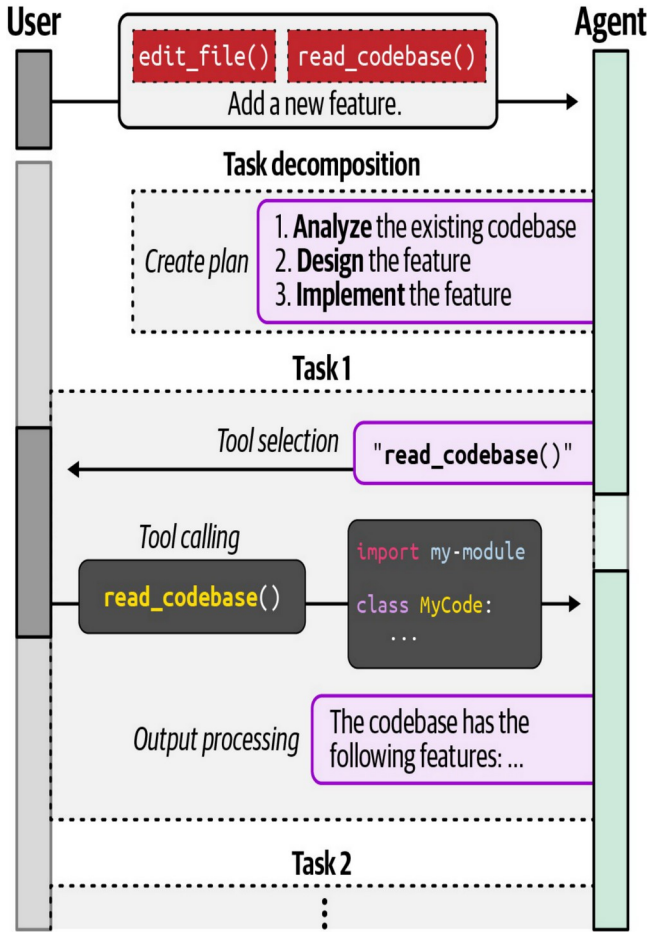
The plan comes first. Then each task or subtask can be processed with tools. The tasks may even name the tool that the agent will use.

Example

The agent makes a three-step plan for adding a feature. The first task, analyzing the codebase, is done with a tool call that retrieves the code.

Update the plan

After analyzing the code and reporting its core features, the agent may update the plan and continue to the next task.



Grootendorst & Alammar (2026). Fig. 6-3, PDF p. 300

Chain-of-Thought is task decomposition

Step by step

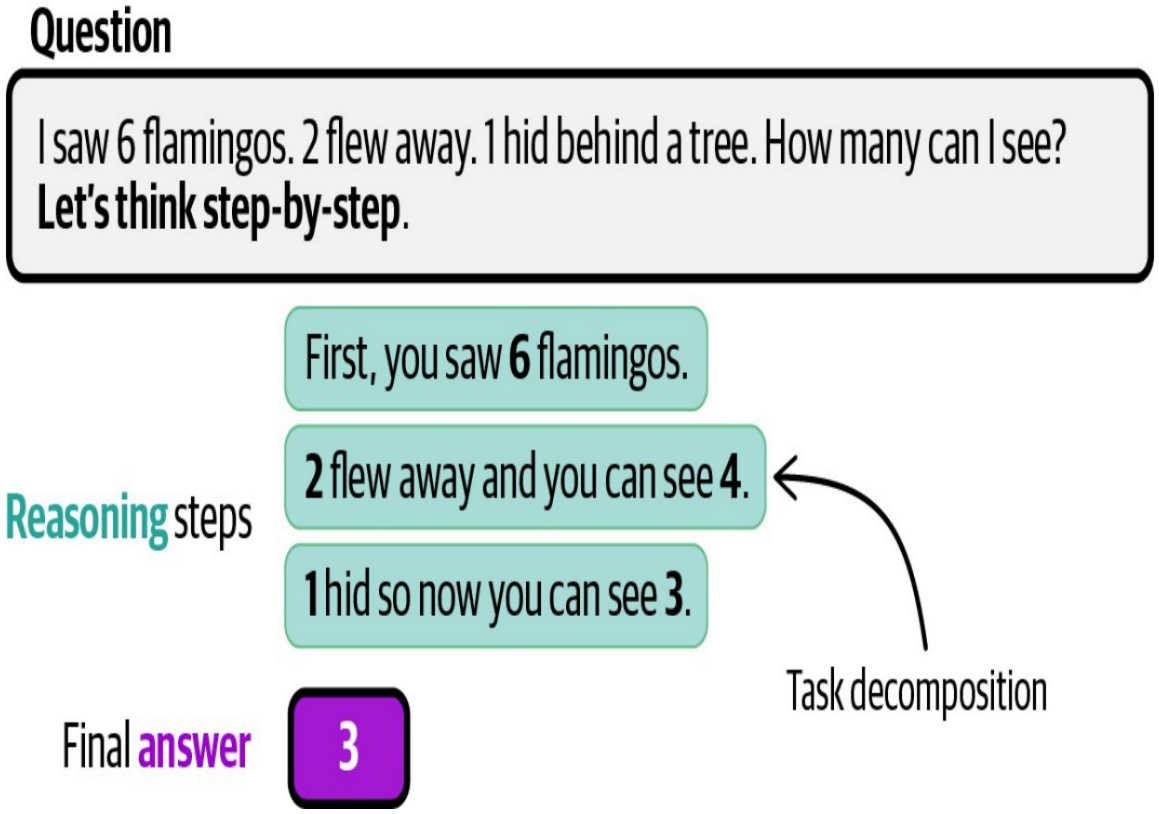
Chain-of-Thought (CoT) asks an LLM to solve problems step by step, either with examples (few-shot) or by stating "Let's think step-by-step" (zero-shot).

Sequential substeps

Each reasoning trace relies on the ones before it. The original query is broken into sequential reasoning substeps.

More than thinking aloud

CoT was meant to scale test-time compute, but the thinking can also feed the plan. CoT is a flexible technique for decomposition.



Grootendorst & Alammar (2026). Fig. 6-4, PDF p. 301

From a reasoning trace to a plan

The prompt

The user asks the model to add a new feature to an existing codebase and ends with "Let's think step-by-step".

The reasoning trace

The model thinks aloud: clarify the requirements, analyze the codebase, design the feature, implement it without changing existing behavior, test it, and update the documentation.

The plan

The same steps become a checklist of six unchecked items, from clarifying requirements to updating documentation.

Reasoning LLMs

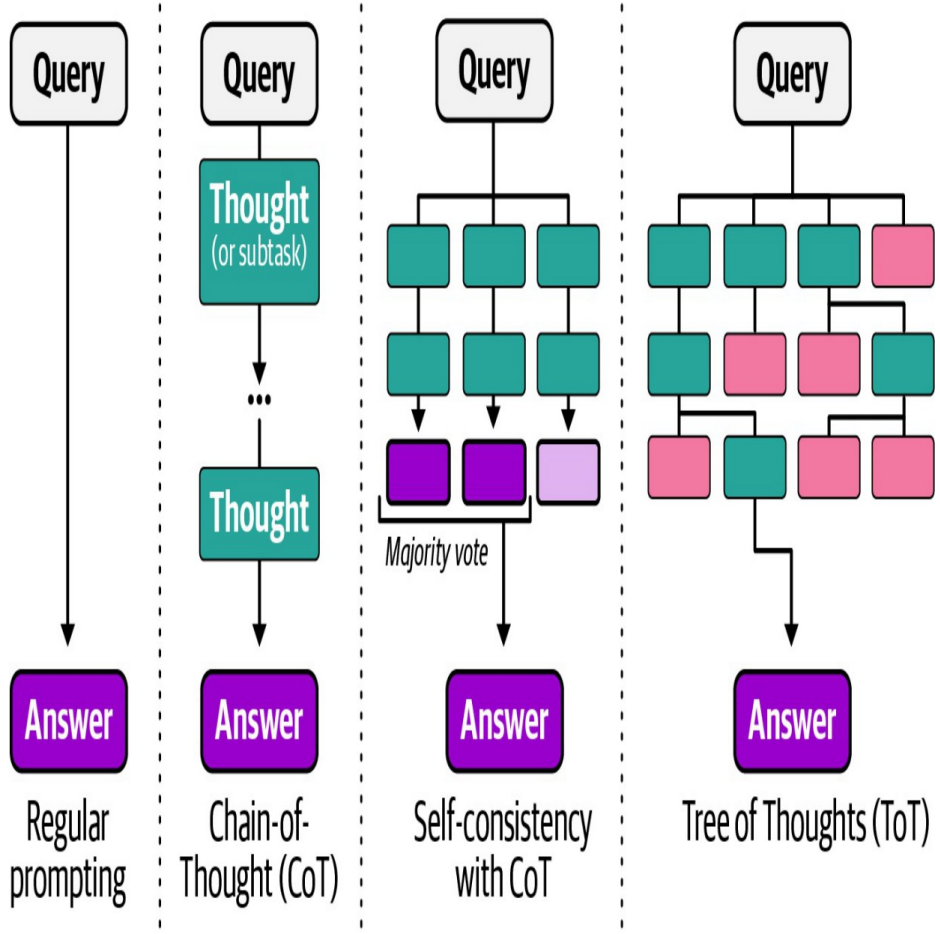
A reasoning LLM first thinks about the steps needed to resolve the query. That reasoning trace is what it leverages to create the plan.

Self-consistency and Tree of Thoughts

Many traces
Self-consistency and Tree of Thoughts sample multiple CoT traces from the LLM in order to find the most optimal one.

Self-consistency
The task is broken down like CoT, but multiple diverse paths are generated and the most consistent answer is selected.

Tree of Thoughts
It explores multiple reasoning paths, where each node is a task or thought, instead of one linear chain. Beam search or Monte Carlo Tree Search with reward models can choose the best path.



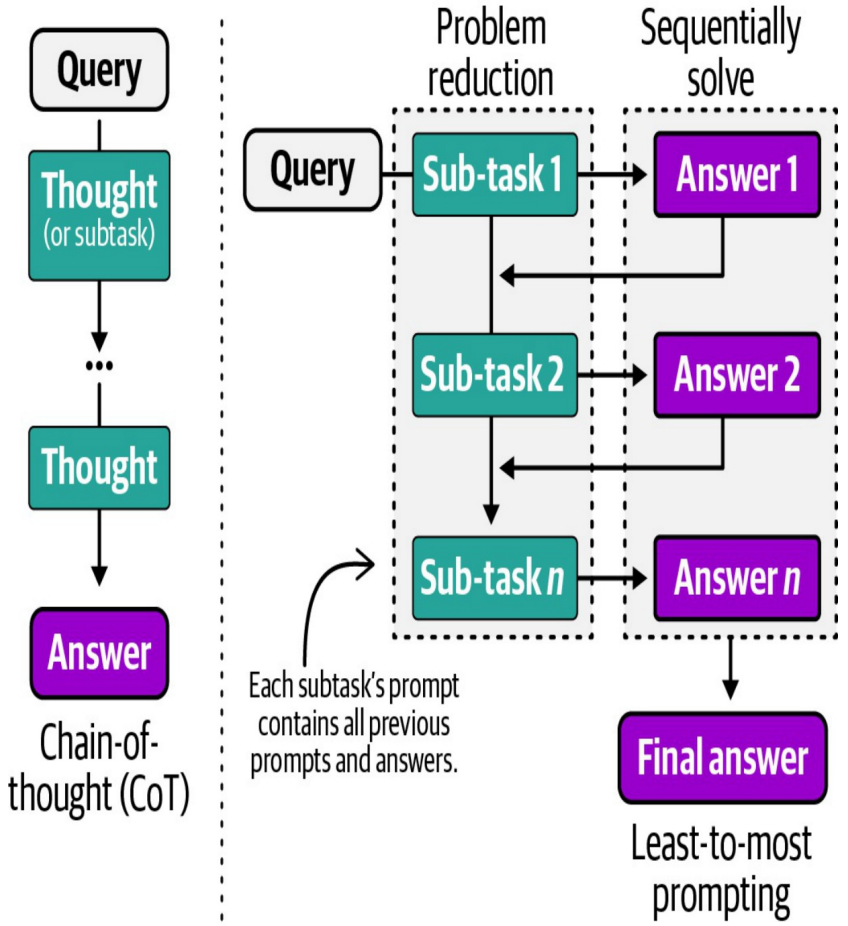
Grootendorst & Alammar (2026). Fig. 6-5, PDF p. 303

Least-to-most prompting decomposes first

Explicit planning
Least-to-most (LtM) prompting was released in 2022 and builds on CoT. It first decomposes the problem into subproblems, then solves them one by one.

Feeding answers forward
Unlike CoT, the solution of each subproblem is fed back into the LLM when it attempts the next one.

Building on each other
The subproblems build upon each other, so later ones can use what earlier ones established.



Least-to-most on a math word problem

Stage 1: problem reduction

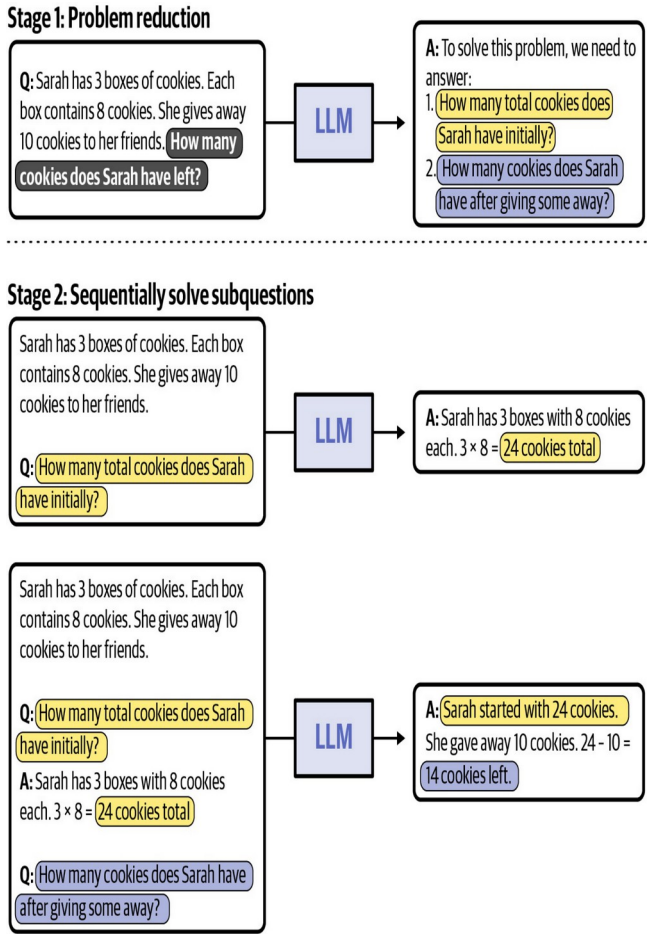
The original problem is decomposed into a set of manageable tasks, here how many cookies Sarah has initially and how many remain after she gives some away.

Stage 2: solve in order

Each subtask is solved after the other. After each one, the previous subquestion and its answer are appended to the prompt.

Few-shot examples

Few-shot prompting shows the model how to decompose the task and how to solve the pieces, which creates this behavior.



Grootendorst & Alammar (2026). Fig. 6-7, PDF p. 306

Plan-and-solve is a zero-shot alternative

Extends zero-shot CoT

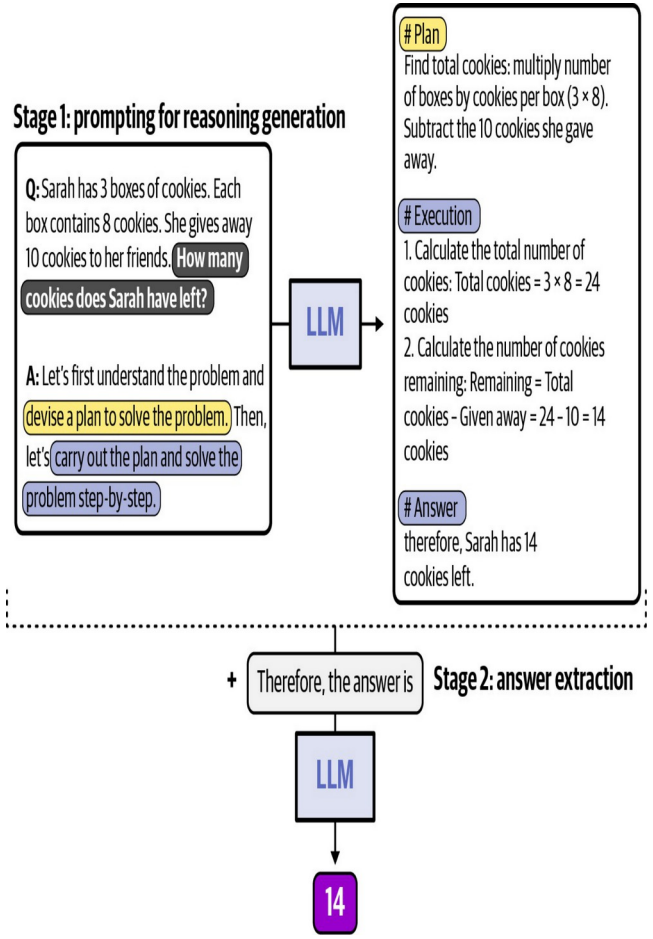
Instead of "Let's think step-by-step", the LLM is asked to devise a plan first and then solve the problem step by step.

Two steps

Prompting for reasoning generation: the model writes a plan and carries it out. Answer extraction: the output is processed into the final answer.

One reasoning step

In the figure the plan and the execution happen within a single reasoning step, followed by extraction of the number.



Why the Q/A format and what changed

Early LLMs

These prompts use a question and answer structure. In the early days, LLMs were mostly trained to autocomplete the prompt, so extra steering was needed to make them answer a question.

The same idea, implicitly

The output is essentially the thinking process that we know from reasoning LLMs. It works just the same for regular LLMs, but the process is implicit rather than visible.

Today

Most LLMs already create advanced plans without complex instructions or specialized prompts. As Chapter 3 explained, they often need to be trained on CoT-like or planning-like reasoning traces to plan well.

Comparing the decomposition methods

CoT
One linear trace. The model thinks step by step. Cheap, and works zero-shot or few-shot, but one path may be wrong.

Self-consistency and ToT
Many traces or a tree of paths, then a selection. More compute for a more reliable choice.

Least-to-most
Decompose first, then solve in order and feed each answer forward. Needs few-shot examples.

Plan-and-solve
A zero-shot plan, then execution in one pass. Simple to apply, but nothing checks the plan along the way.

Teaching addition. Concepts: Ch. 6, PDF pp. 298–309

From planning to action sequencing

Not yet agents

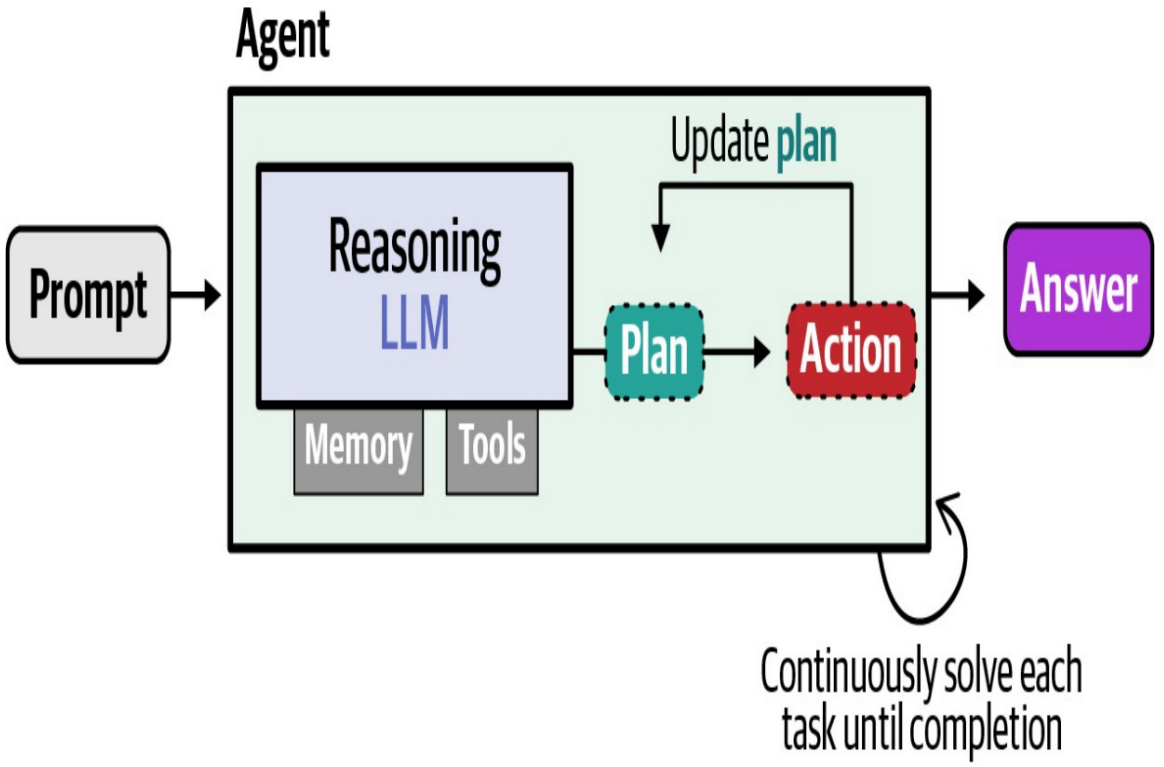
LtM and plan-and-solve decompose a problem into subtasks that are solved in order. That works for LLMs but not yet for agents, which sequence actions autonomously.

Action sequencing

After deciding its subgoals, the agent determines a sequence of actions that move it from its current state to the goal state.

Efficiency

A coding agent that skips reading the codebase is inefficient and redundant. A well-thought-out sequence reduces wasted resources and time.



Grootendorst & Alammar (2026). Fig. 6-9, PDF p. 310

Before ReAct: reasoning and acting were separate

Two capabilities

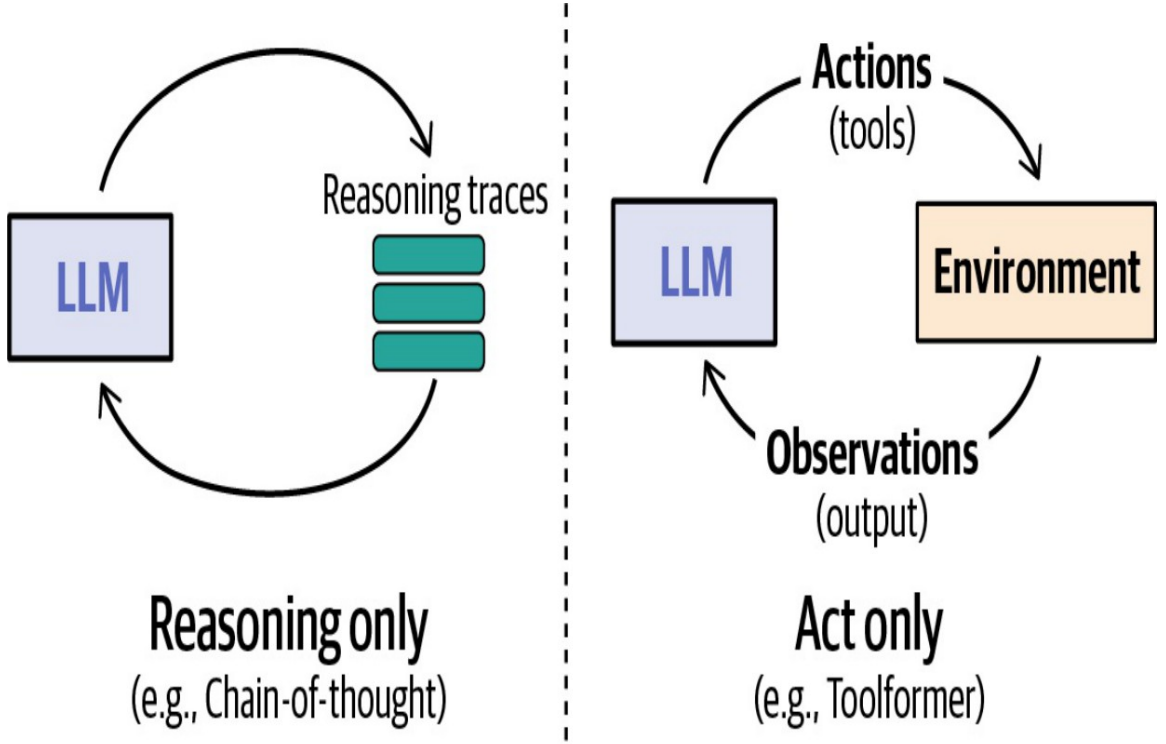
Reasoning could be enabled with CoT, and LLMs could take actions, such as with Toolformer from Chapter 5. The two were treated as separate.

Hard to iterate

With reasoning and acting separate, it is difficult for an agent to iterate, because the result of an action cannot easily change the next thought.

The need

What was needed is a technique that sets an initial plan and keeps updating the action sequence as results arrive.



Grootendorst & Alammar (2026). Fig. 6-10, PDF p. 311

ReAct fuses reasoning and acting

Reason and Act

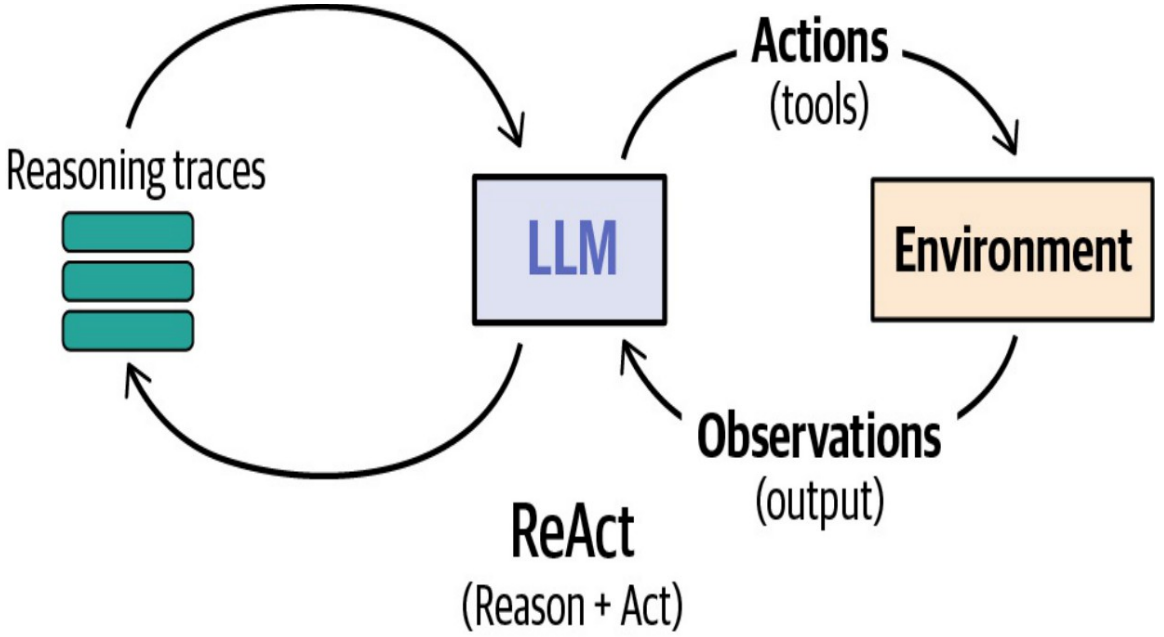
ReAct takes inspiration from CoT for reasoning and from tool use for acting and combines them. It interleaves reasoning traces with task-specific actions.

Feedback loop

The model cycles through thought, action, and observation. In each loop its output is separated into those three components.

First autonomous systems

The book calls the result the first truly autonomous systems that drive AI agents.



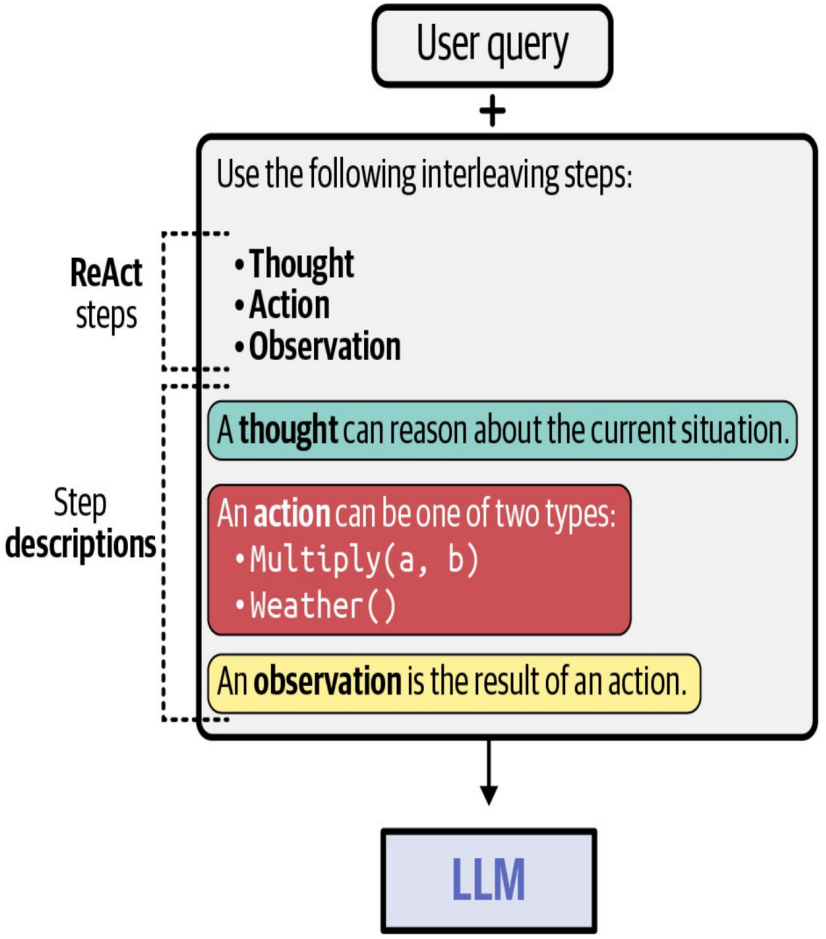
Grootendorst & Alammar (2026). Fig. 6-11, PDF p. 312

ReAct can be enabled through prompting

Three entities
The model is prompted to write a thought, an action, and an observation as separate parts of its output.

Instruction following
If the model follows instructions well, the format alone can produce ReAct. Few-shot examples of cycles are typically added to match the behavior.

Stopping
The LLM stops when it reaches a predefined goal. Long runs need memory and context engineering from Chapter 4.



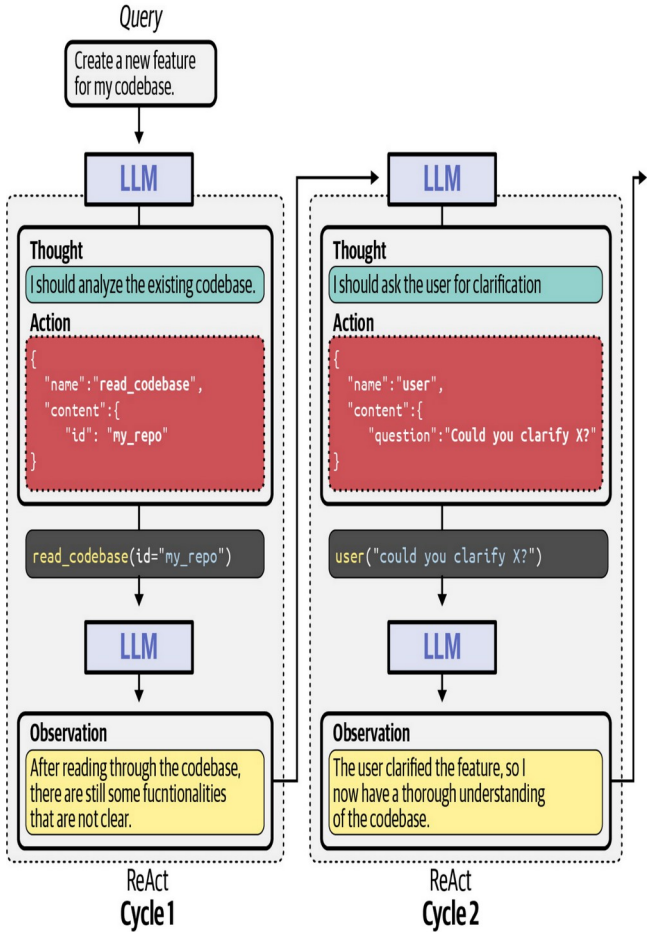
Grootendorst & Alammar (2026). Fig. 6-12, PDF p. 313

Two cycles of thought, action, and observation

Cycle 1
The thought says the agent should analyze the codebase. The action calls read_codebase on the repository, and the observation notes that some functionalities are still unclear.

Cycle 2
The next thought is to ask the user for clarification. The action is a message to the user, and the observation records the answer.

Adjusting
By iterating over thoughts and observations, the LLM plans actions, observes the output, and adjusts accordingly.



Grootendorst & Alammar (2026). Fig. 6-13, PDF p. 315

Building ReAct into TinyAgent: the ReAct class

Prompting the format

ReAct mostly needs careful prompting. The book's ReAct class provides a prompt that asks for exactly one step per turn, in a THOUGHT and ACTION format.

Final answer as an action

Finishing also uses an action. A final_answer tool carries the answer, and it is the only way to complete the task, so the agent does not get stuck in a loop.

max_steps

A parameter sets the maximum number of steps. The agent can stop early, but this prevents it from looping forever.

Parsing

A parse function splits the model's text into the THOUGHT and the ACTION and stores them as the response's reasoning and content.

Grootendorst & Alammar (2026). Code, PDF pp. 316–318

The autonomy loop is a for-loop

Three changes

To make TinyAgent autonomous, the book adds the ReAct instruction to the system prompt, an autonomy loop to the run function, and parsing of THOUGHT and ACTION.

A for-loop

The most important step is a loop: call the LLM repeatedly until it decides to stop, or until the maximum number of steps is reached.

One step

Each step generates a response, stores it in memory, parses it, stops if the final answer was given, and otherwise runs the chosen tool.

Observation to memory

The tool result is added to memory as an observation, and the step is recorded in a trajectory object for later inspection.

Running the agent on a multi-step calculation

The task

A small model with no native thinking or tool calling is given add, subtract, and multiply tools. It is asked for (4.6 + 6.685) × 4, minus 3.14.

Step 1: add

The thought already lays out a plan of three operations. The first action adds 4.6 and 6.685 and the observation is 11.285.

Steps 2 and 3

The agent multiplies 11.285 by 4 to get 45.14, then subtracts 3.14. Each thought reflects on the previous action and continues.

Step 4: answer

The agent sees that all calculations are done and returns 42.0 with the final_answer tool. The trajectory shows exactly how.

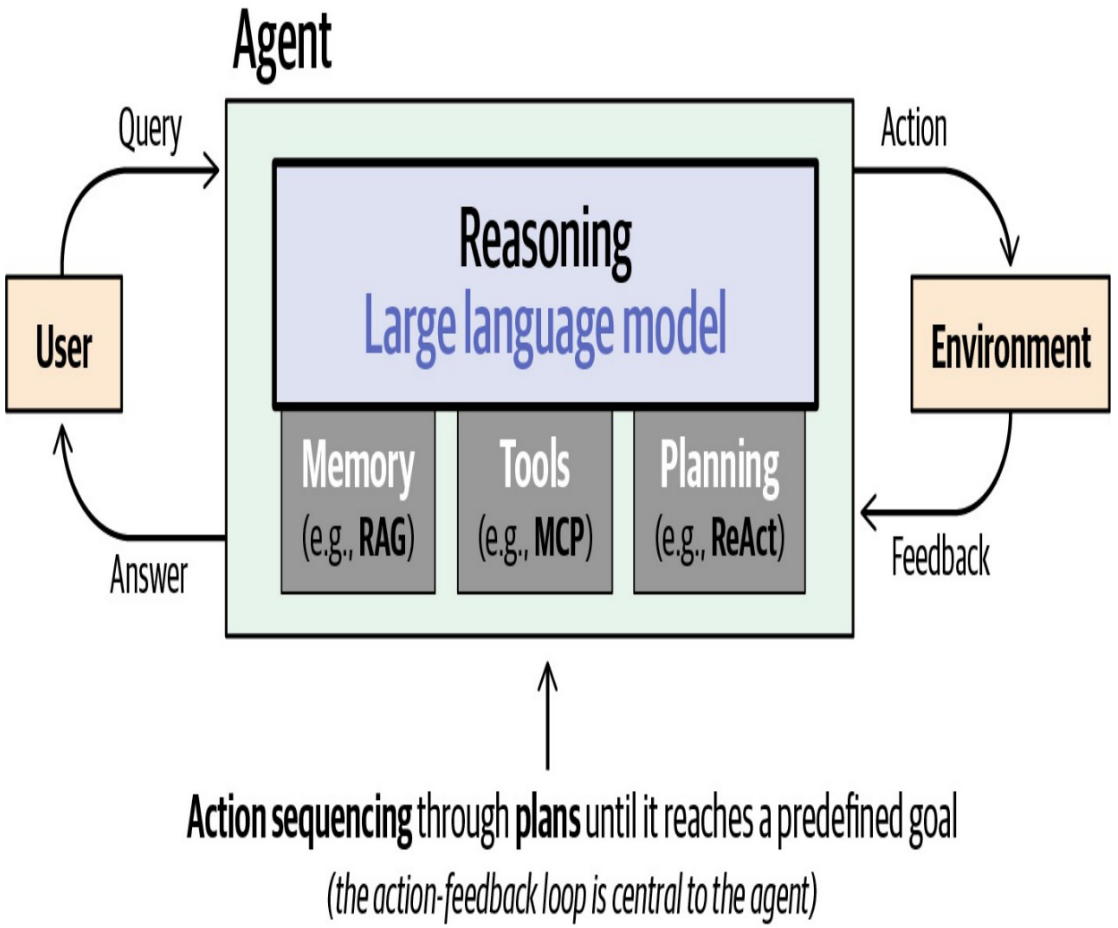
Grootendorst & Alammar (2026). Code, PDF pp. 319–321

The components of an autonomous agent

Four components
A reasoning LLM as the brain, tools to interact with the environment, memory so the LLM does not forget past actions, and planning with frameworks such as ReAct.

Run it continuously
When these are combined and the system runs continuously, like a while-loop or for-loop, you have an agent.

Multi-turn tools
Chapter 5's tools were single-turn. ReAct-like frameworks turn them into multi-turn processes that use memory, tools, and planning.



Grootendorst & Alammar (2026). Fig. 6-14, PDF p. 322

Reading a ReAct trace

What to look for

For each step, read the thought, then the action, then the observation. Check that the next thought uses the previous observation. Check the final answer is given with the stopping action.

Failure patterns

The agent repeats the same action, ignores an error observation, calls a tool that does not exist, or never calls the final answer tool. The step limit exists to cut such loops short.

Why it matters

A trajectory is evidence of how a result was produced. For high-stakes work, the process and the answer should both be reviewed, which the later chapter on evaluation develops.

FireAct: fine-tuning on ReAct trajectories

Why fine-tune

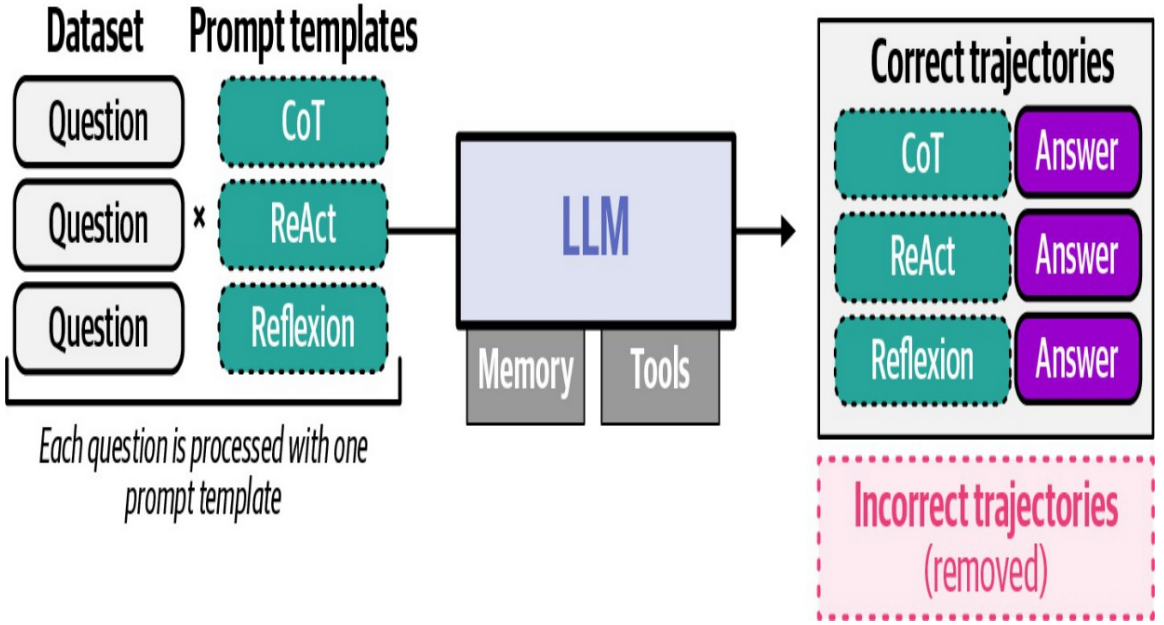
Prompt-based ReAct needs few-shot examples, which waste tokens in the context and are hard to get right. FireAct instills ReAct into a small LLM by fine-tuning.

Step 1: generate

GPT-4 generates trajectories for questions from several datasets, using CoT, ReAct, and Reflexion templates. Only correct trajectories are kept.

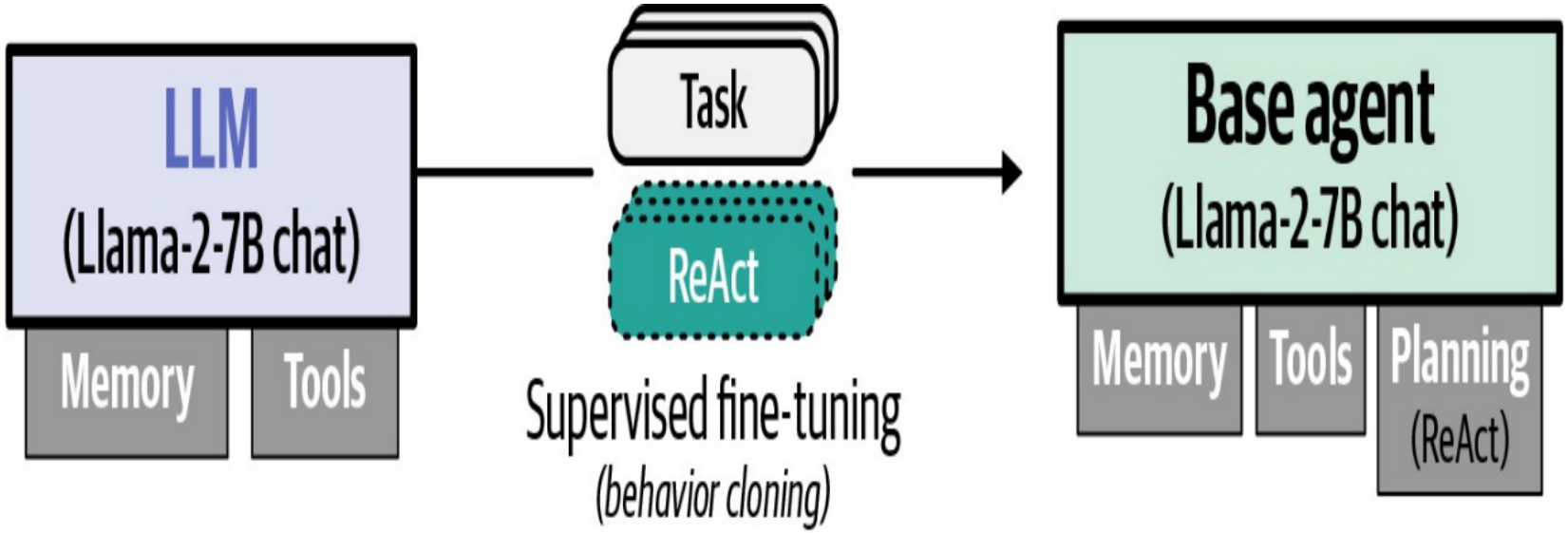
Step 2: fine-tune

Trajectories are converted to the ReAct format and used to fine-tune a smaller model, with full or LoRA fine-tuning. No few-shot examples are needed at inference.



Grootendorst & Alammar (2026). Fig. 6-15, PDF p. 323

Supervised fine-tuning mimics the data



Mimicry
SFT teaches the LLM to reproduce the behavior in the training data. It is limited because the model does not adapt through experience.

No exploration
As Chapter 5 explained, SFT lacks the ability to explore the environment, which tends to lead to suboptimal policies.

Imitation learning
In ETO's terms SFT is behavior cloning. It is often called imitation learning because the model copies shown behavior.

Grootendorst & Alammar (2026). Fig. 6-16, PDF p. 325

ETO: exploration and optimization with RL

Learn rather than mimic

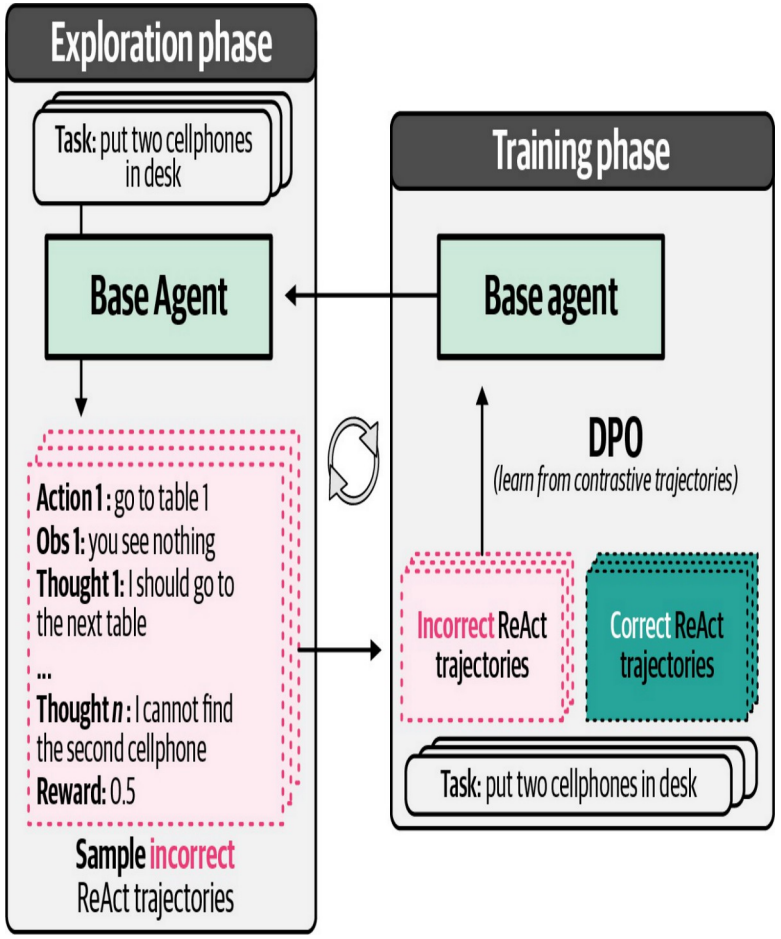
Exploration-based Trajectory Optimization (ETO) uses reinforcement learning to encourage the LLM to explore trajectories rather than copy behavior.

Exploration phase

The base agent interacts with the environment. Failed trajectories are sampled and paired with correct ones previously collected for the same task.

Optimization phase

Pairs are used to fine-tune the model with Direct Preference Optimization (DPO), raising the likelihood of successful trajectories and lowering that of failed ones.



Grootendorst & Alammar (2026). Fig. 6-17, PDF p. 326

ETO details: ALFWorld, SFT first, RL second

A household environment

ALFWorld is a text-based environment that mimics a household. Tasks include cleaning a tomato and putting it on the countertop, or examining an alarm clock with a desk lamp.

Actions and observations

The agent issues text commands such as going to the desk, taking an object, or using the lamp. The environment answers with a text observation, until the task is won.

Why SFT then RL

SFT teaches the right behaviors and format. It gives the foundation the model needs to explore and improve itself with RL.

No prompt needed

A major benefit of SFT or RL, or both, is that ReAct no longer has to come from prompting.

Grootendorst & Alammar (2026). PDF pp. 324–326

Native ReAct: reasoning and tool calling built in

Prompted ReAct is brittle

The THOUGHT, ACTION, OBSERVATION scheme relies on instruction following and takes a fair bit of the system prompt. Training a model to do it natively is the answer.

Choose a capable model

The book switches to a model with native thinking and tool calling, so the model produces reasoning and tool calls in its own fields.

Mapping

THOUGHT becomes the response's reasoning. ACTION becomes its tool call. OBSERVATION is the tool output added to memory with the tool role.

NativeReAct

The class removes the ReAct prompt and the parsing. All that remains is the maximum number of steps.

Native ReAct run and what TinyAgent became

Same task, same answer

With the native version the agent answers 'The result is 42.0.' The loop is the same: it stops only when the model makes no tool call.

Each step's choice

At every step the model can choose to reason, to call a tool, or to give a final answer, recorded in separate response fields.

Fewer visible thoughts

In the trajectory the first step has reasoning and a tool call, the next two only tool calls, and the last reasoning and the answer. Each LLM behaves differently.

What we built

TinyAgent now has an autonomy loop, a planning module with ReAct, memory, tools, and a trajectory. The book says its true foundation is complete.

Agents that continuously improve

Self-generated feedback

Instead of relying on training phases or reward models, agents increasingly use self-generated feedback loops to improve as they act.

Not all feedback is equal

Feedback prevents local minima and points to better solutions. But continuous self-improvement is tricky for LLMs that are fundamentally static when they act.

Reflection

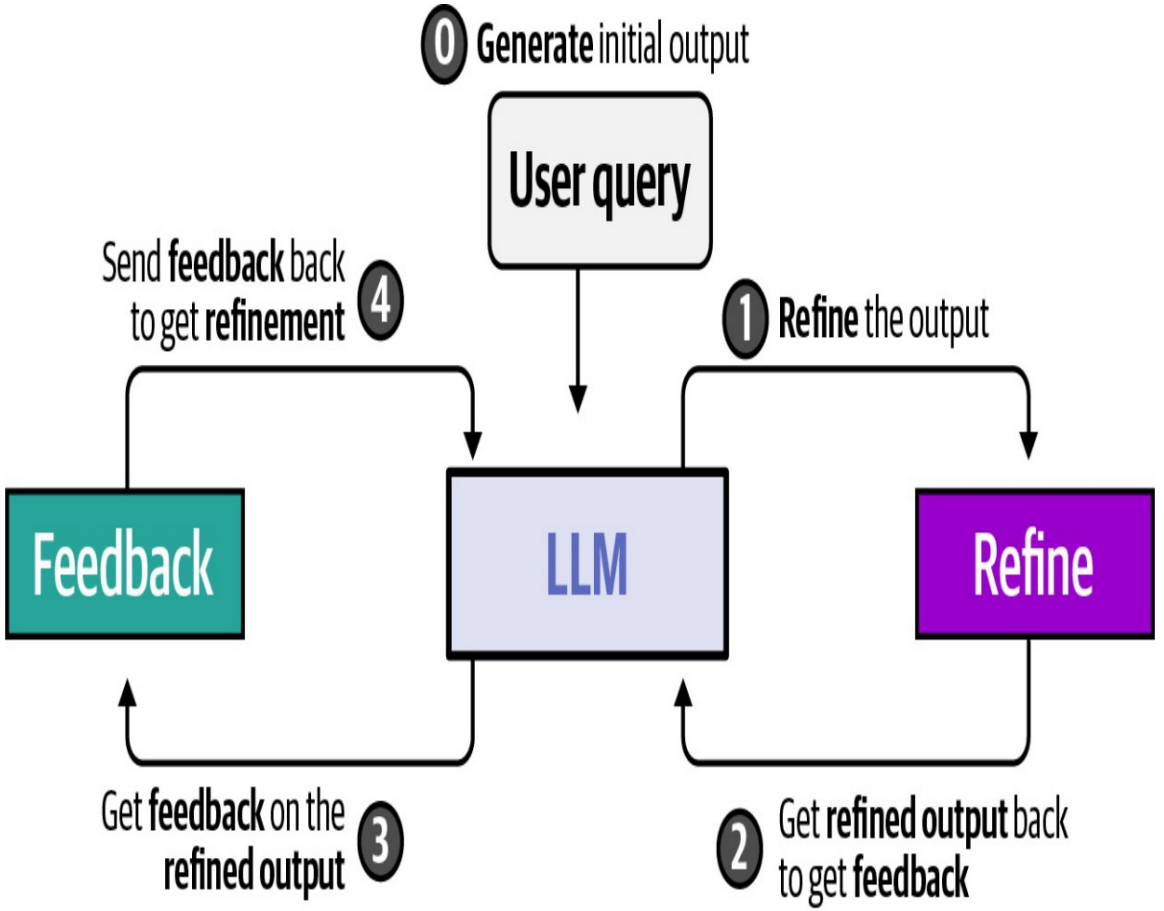
Environment feedback says what happened after an action. Reflection is typically an internal process in which the LLM is critical of its own output, often together with external sources.

Self-Refine: the LLM as its own editor

Generate, critique, refine
The LLM writes an initial answer, critiques it (feedback), and then refines it based on that reflection. The cycle repeats.

Same LLM
The same model generates the initial output, the feedback, and the refined output, hence the name Self-Refine.

Stopping
The loop ends at a stopping criterion, such as a number of steps or an LLM-guided stopping mechanism. It was inspired by how people draft and revise.



Grootendorst & Alammar (2026). Fig. 6-18, PDF p. 332

A Self-Refine example

An example from the paper

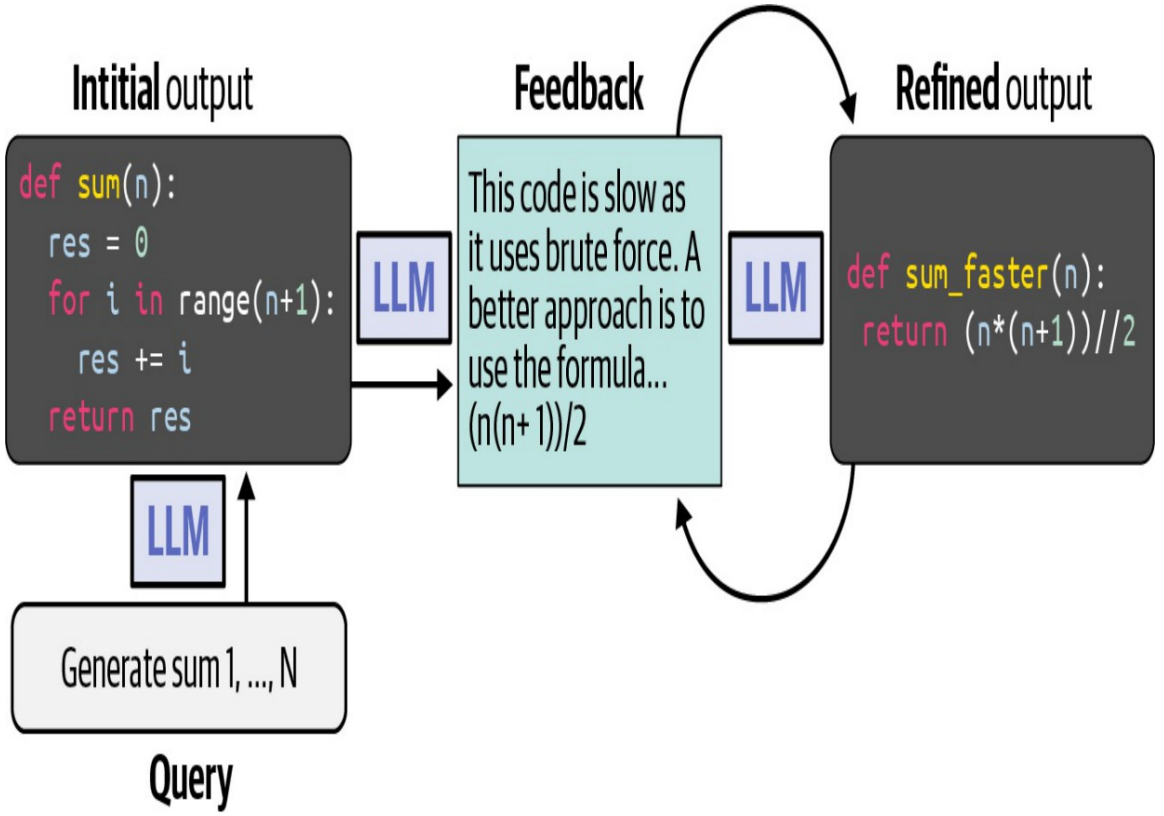
The figure starts from a generated output and then shows an iterative loop of first feedback and then a refined output.

Improving step by step

Each pass identifies what is wrong or missing and rewrites the output to fix it, so the final output is the product of several passes.

Not yet agentic

The framework is about refining an LLM's own output and does not mention an agentic system. Reflexion extends the feedback idea to ReAct trajectories.



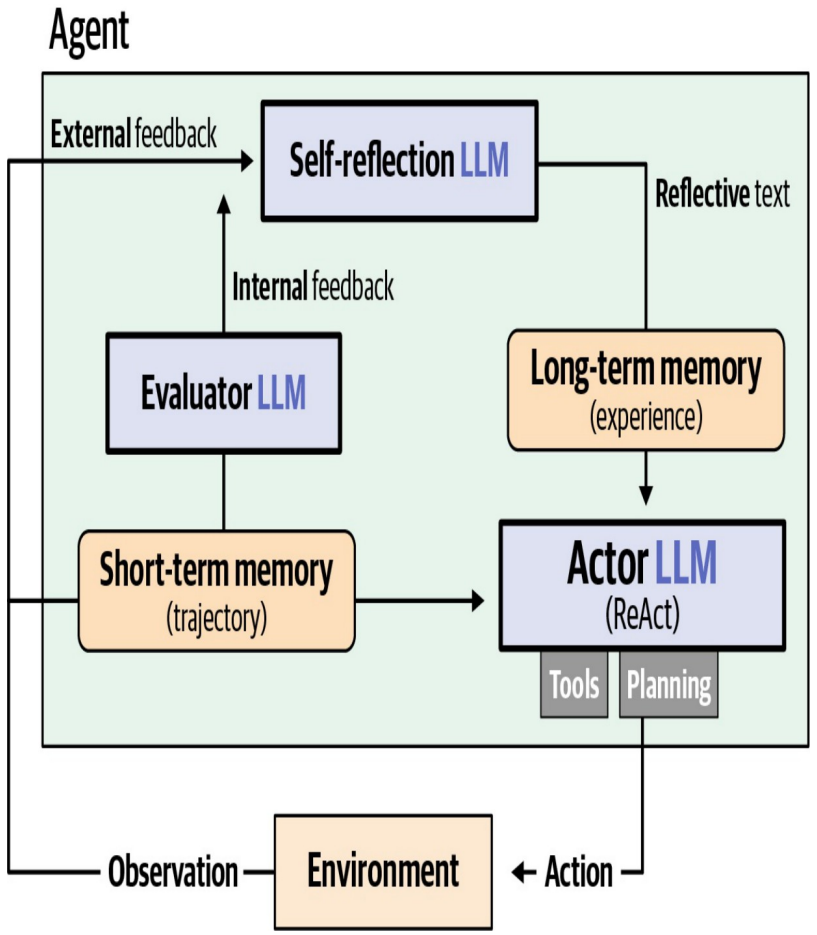
Grootendorst & Alammar (2026). Fig. 6-19, PDF p. 333

Reflexion: three LLM roles

Actor
A ReAct LLM that executes actions based on observations and trajectory data. It is the main brain of the agent.

Evaluator
It assesses the quality of the trajectory and output by asking how well the Actor did, and gives a scalar reward.

Self-Reflection
It writes more specific feedback from the full trajectory, including the Evaluator's output, as a reflective summary for the Actor to use.



Grootendorst & Alammar (2026). Fig. 6-20, PDF p. 335

Reflexion memory, CRITIC, and the limits of prompting

Two kinds of memory

The trajectory is kept in short-term (conversation) memory, and the reflective experiences based on trajectories are kept in long-term memory, as episodic memory.

Better next time

These verbal reflections on observations are kept so that decision making improves in subsequent trials.

CRITIC

It starts with an initial output and revises it using external tools, such as search, to obtain more information. Feedback can come from the agent itself or from the environment.

The limit

Prompt-based techniques only guide behavior and do not instill it in the model's parameters. Fine-tuning can make reflection part of the agent's nature.

Grootendorst & Alammar (2026). PDF pp. 334–336

Self-improvement through reinforcement learning

Beyond prompting

Prompt-level heuristics matter, but more fundamental change is needed to create a self-sustaining loop of reflection, reasoning, and task generation.

RL as the engine

RL has been shown to enable seemingly unbounded self-improvement with limited need for human-labeled data. RISE adds introspection steps to improve outputs by iterating over attempts.

The labeling problem

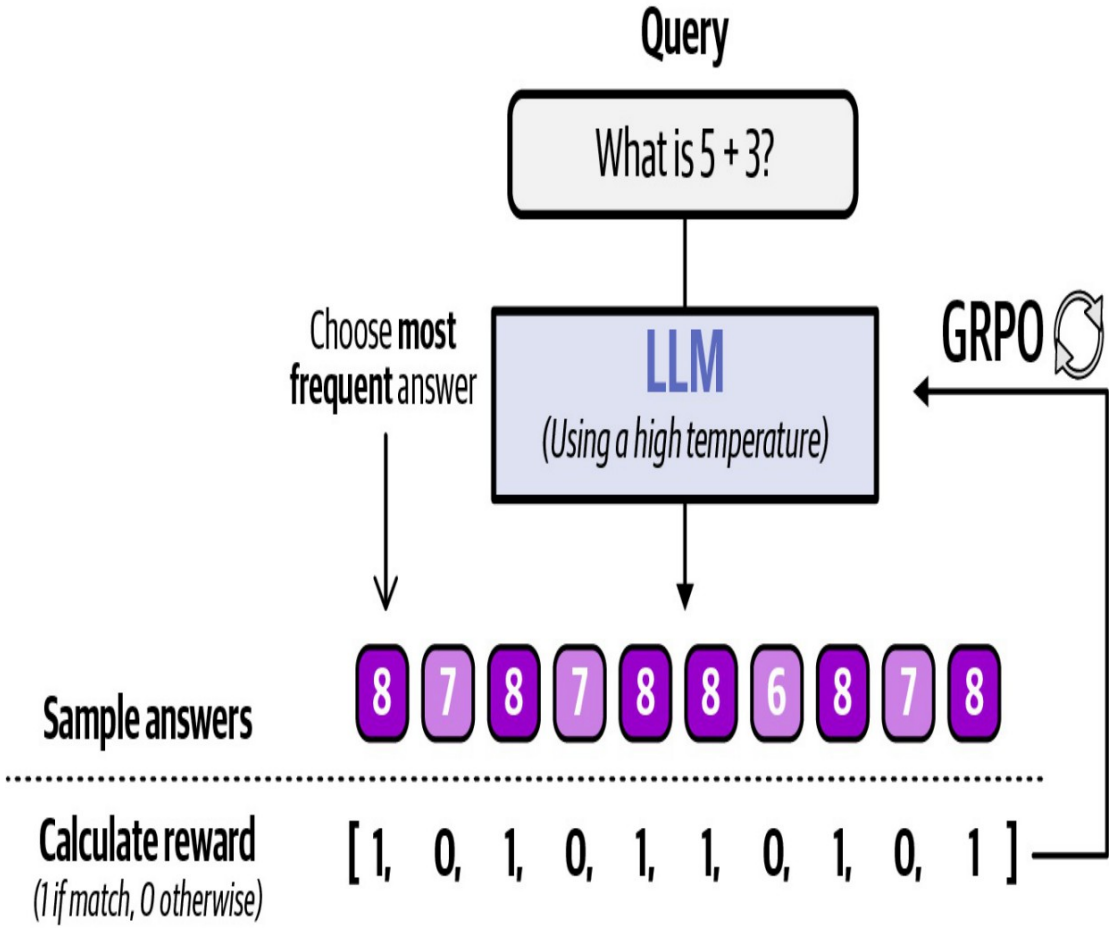
Labeled agentic traces that generalize are hard and costly to create. Self-play needs supervised data, such as query, reasoning, and answer triplets, that is hard to collect for complex real-world tasks.

Test-Time Reinforcement Learning (TTRL)

Learn at test time
TTRL updates the model's parameters with RL while it interacts with its environment. It needs no supervised data.

Four steps
Sample 16 answers per query with temperature 0.6. Take a majority vote. Reward each answer by how it aligns with the vote. Use the rewards in RL with GRPO.

Reused pieces
Majority voting and GRPO were covered in earlier chapters. Combined, they let the model produce its own rewards.



Grootendorst & Alammar (2026). Fig. 6-21, PDF p. 337

Lucky hits: right signal, wrong majority

Majority can be wrong

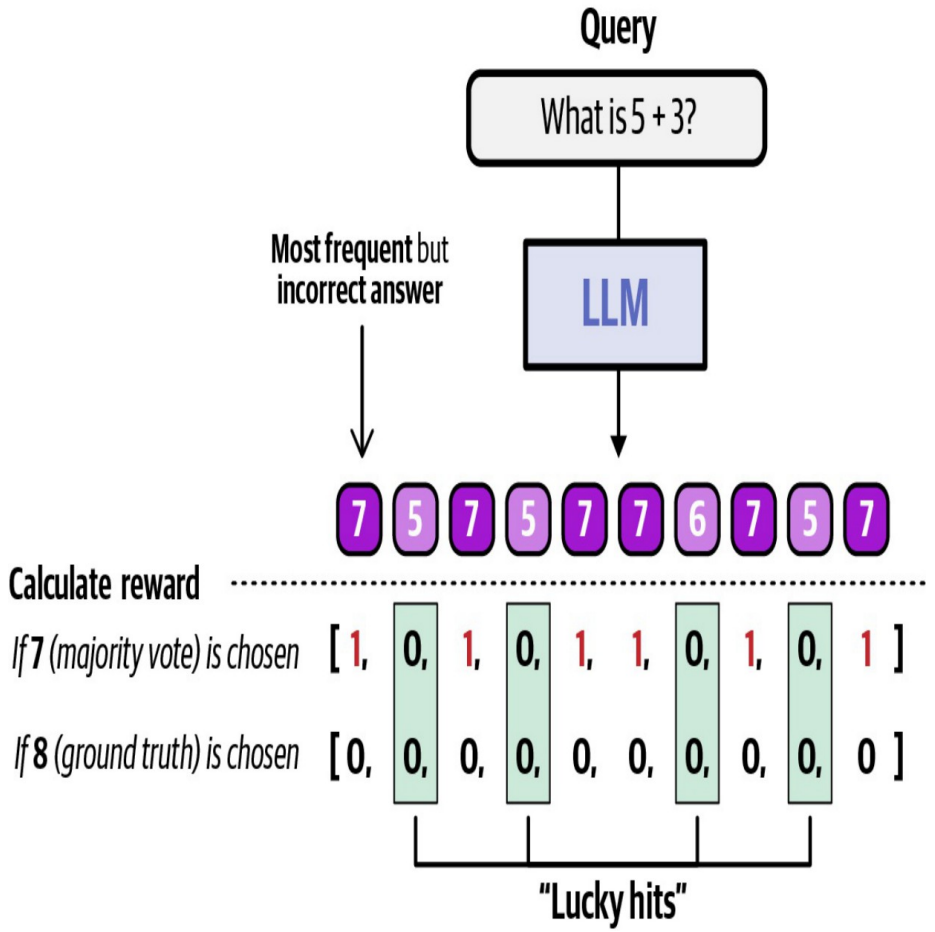
Majority voting is not flawless: the most frequent answer is not always correct.

Vague rewards still help

The authors argue that rewards can be vague, because they signal the need for further exploration instead of continual exploitation, which keeps models out of local minima.

Lucky hits

Even if the majority is incorrect and there are no correct predictions, a wrong answer can still receive the correct negative reward. These are lucky hits.



Grootendorst & Alammar (2026). Fig. 6-22, PDF p. 338

R-Zero: a Challenger and a Solver

Two models, one base

Both models start from the same base model. They coevolve and challenge each other's outputs, with no labeled data.

Challenger

It generates synthetic queries that are hard for the Solver. It is trained with RL (GRPO) from the rewards that the Solver gives it.

Solver

It samples several answers and selects the best by majority vote, as in TTRL. While the Challenger is trained, the Solver is frozen and serves as a reward model.

Rewards and penalties

An uncertainty reward (0 to 1), a repetition penalty (0 to 1), a format reward (0 or 1) for question tags, and a composite reward of uncertainty minus repetition, floored at 0.

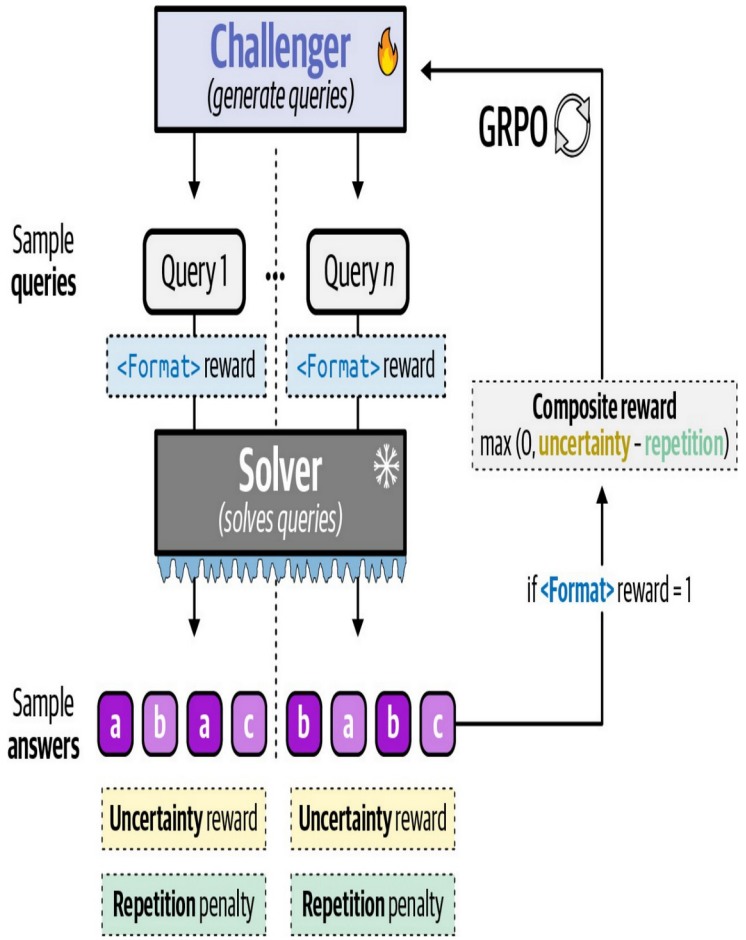
Grootendorst & Alammar (2026). PDF pp. 338–339

Training the Challenger

Signal
The Challenger is trained by trying to give the Solver more difficult queries. Its signal is whether it can produce one.

The Solver as judge
The Solver samples answers to each query. Its agreement rate gives the uncertainty reward, and similar queries are penalized.

Frozen Solver
The Solver is not updated in this phase. It is only used to score the Challenger's queries.



Grootendorst & Alammar (2026). Fig. 6-23, PDF p. 340

Training the Solver

Curate the data

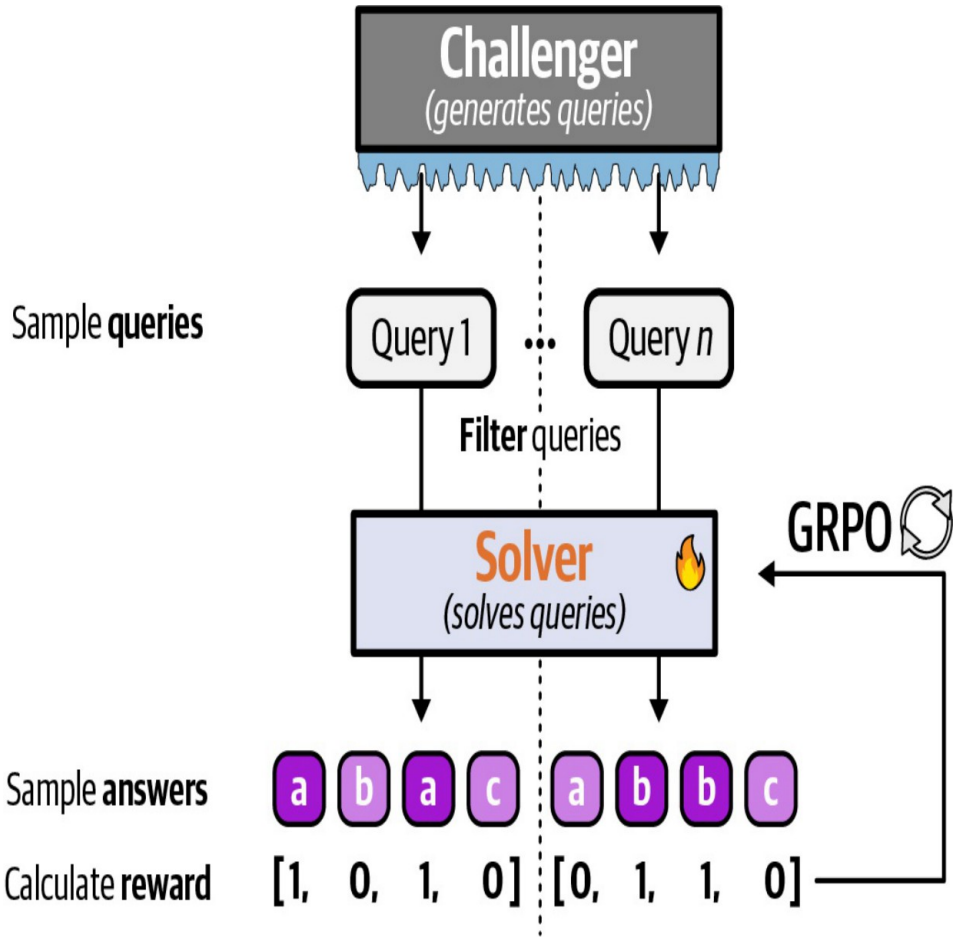
Queries sampled from the Challenger are filtered. Those with too few correct answers are too hard, and those with almost all correct are too easy. Both are dropped.

Just difficult enough

The Solver is fine-tuned on the curated queries that are just difficult enough.

Binary reward

GRPO is used again. The reward is simply 1 for a correct answer and 0 otherwise, a verifiable reward akin to TTRL.



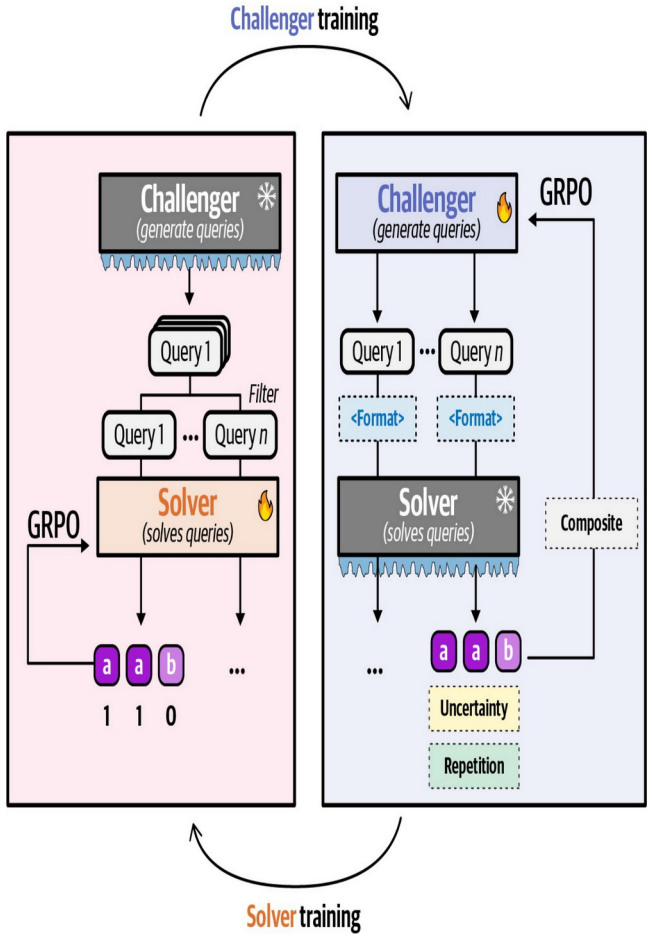
Grootendorst & Alammar (2026). Fig. 6-24, PDF p. 341

A coevolving loop with asymmetric rewards

Take turns
Challenger training and Solver training run after each other. The Challenger makes harder queries and the Solver gets better at solving them.

Asymmetry
The Challenger gets a continuous reward, from 0 to 1, so it explores a range of difficulties. The Solver gets a binary reward, so it learns to be precise.

Competition
Each tries to win, which creates an incentive for competition and a curriculum at the edge of difficulty.



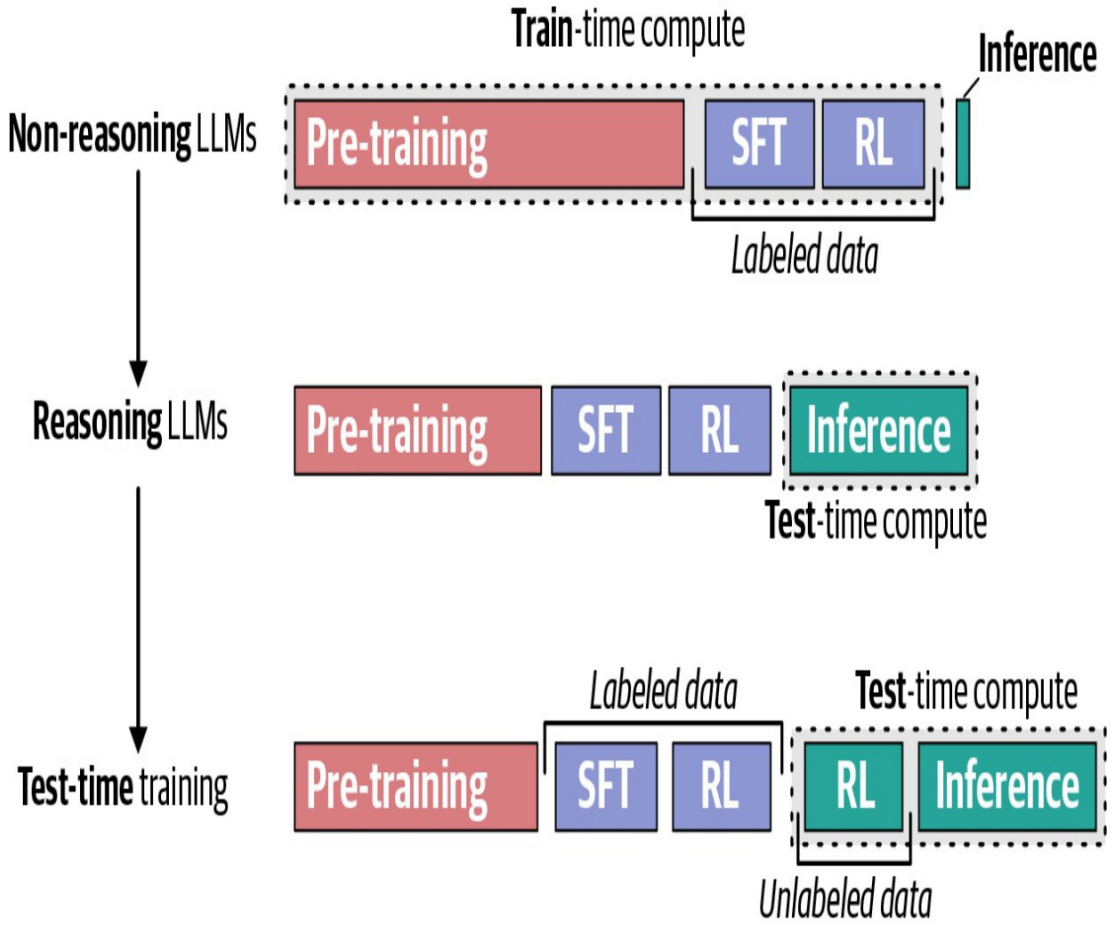
Grootendorst & Alammar (2026). Fig. 6-25, PDF p. 343

Test-time training as a new paradigm

Newer techniques
TTRL and R-Zero are, at the time of writing, newer methods with great potential for self-improving agents.

Potential shift
The training focus moves toward unlabeled data and inference rather than only pre-training combined with SFT and RL.

Compute at inference
Test-time training moves more compute to inference, where existing models can be scaled more easily.



Grootendorst & Alammar (2026). Fig. 6-26, PDF p. 344

Three ways to shape agent behavior

Prompting
ReAct, Self-Refine, and Reflexion. Fast to try and needs no training, but costs context tokens and can be brittle.

Supervised fine-tuning
FireAct. Copies correct trajectories, so no few-shot examples are needed. The model imitates and does not explore.

Reinforcement learning
ETO uses success and failure pairs to learn from experience. It needs an environment and a way to judge outcomes.

Self-improvement
TTRL and R-Zero generate their own signals, with the model improving as it acts. They are newer and less proven.

Teaching addition. Concepts: Ch. 6, PDF pp. 311–344

Activity: plan and reflect on a research task

The task

An agent must prepare a one-page briefing on a topic for a department meeting. In pairs, write the plan as a numbered checklist of subtasks, as in the chapter's six-step example.

Run the loop

Write three thought, action, observation cycles for the first steps, naming the tool for each action. Mark where the plan must change because an observation was surprising.

Reflect

Add an evaluator that scores the draft and a reflection that says what to change. State what would be written to memory so the next attempt is better, and when the agent should stop.

Practical cautions for planning and reflection

Bound the loop

Set a maximum number of steps, as the book's ReAct class does, and a clear final-answer action. An agent without a stop rule can loop.

Mind the context

Long runs fill the context with earlier steps. Memory and context engineering from Chapter 4 decide what the agent still sees.

Do not trust the majority blindly

Self-consistency and TTRL rely on voting, and the majority is not always correct, as the chapter notes for lucky hits.

Keep humans in view

Self-improving agents change themselves. Review the trajectories, because the process matters and not only the answer.

Teaching addition. Concepts: Ch. 6, PDF pp. 313–344

Learning check with worked answers

1. How does LtM differ from CoT?
LtM first decomposes the problem and then solves subproblems in order, feeding each solution into the next prompt. CoT produces one linear trace.

2. What does ReAct add?
It interleaves reasoning with actions in a loop of thought, action, and observation, so the agent can adjust its plan as results arrive.

3. SFT versus RL for ReAct?
SFT (FireAct) copies correct trajectories. RL (ETO) learns from success and failure pairs and can explore beyond the data.

4. Why can TTRL work without labels?
It samples many answers, takes the majority as a pseudo-label, rewards agreement, and updates the model with RL (GRPO).

Teaching addition. Concepts: Ch. 6. Cross-reference: Ch. 3, 5, 7

Chapter summary

Plan

Task decomposition breaks complex problems into small ones. CoT-like techniques, least-to-most, and plan-and-solve are used for explicit planning.

Act in sequence

ReAct is the final missing link for autonomy: it decides on its own when to stop and how long to continue. Prompting, SFT, and RL can all enable it.

Reflect and improve

Reflection prevents agents from getting stuck in local minima. Test-time training is a new paradigm in which agents improve as they interact with their environment.

Core distinctions and source guide

Conceptual distinctions

Planning decides what steps to take, and action sequencing carries them out one at a time. ReAct interleaves thought, action, and observation. Self-Refine and Reflexion reflect through prompts, while TTRL and R-Zero change the model's parameters through reinforcement learning.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 6. Selected connections: Chapters 3, 4, 5, 7, and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF